GOAL FIRST — PART TWO

GOAL FIRST IN PYTHON

BUILD YOUR OWN STOCK SCREENER IN PYTHON

A framework for translating your investment logic into code.

2

Leverage Thinking · Laura Kruse-Stecher

Contents

Introduction
The framework, and how to use this guide

Part 1 — From Logic to Code
Pseudocode before Python

Part 2 — Data Quality
Three problems to catch before screening

Part 3 — The Screener
Core function and output

Part 4 — Adding Metrics
Three metrics that matter

Part 5 — Adapting to Your Goal
Dividend and low volatility variants

Conclusion
Build it once, use it forever

Glossary
About the Author · Code & Updates · Imprint

Disclaimer

No Investment Advice

This publication is for informational and educational purposes only. It is not investment advice, not a financial analysis or investment recommendation within the meaning of applicable securities law, not a recommendation to buy or sell any security, and not a solicitation of any kind. It presents a method; investment decisions are yours.

Individual Responsibility

All investment decisions are solely your responsibility. The allocation of capital, the selection of strategies, and the weighting of portfolio components described in this guide are illustrative frameworks – not prescriptions. Your financial situation, risk tolerance, and investment objectives are individual and may differ materially from any example presented here.

Risk Disclosure

Investing in equities involves risk, including the possible loss of principal. Past performance – including all historical examples shown in this guide – is not indicative of future results.

Illustrative Examples and Data

Companies referenced in this guide, including AT&T and Snap, are used solely as illustrative case studies to demonstrate methodological principles. They do not constitute current recommendations or opinions on those securities. Price and return figures are the author's own calculations based on publicly available price data, rounded, as of the time of writing. Figures may differ depending on data source and calculation method.

Author's Positions

The author trades stocks and may hold, or may have held, positions in some of the securities mentioned. Examples were chosen for their illustrative value, not on the basis of the author's positions.

Software and Data

The code examples in this guide are tools for data processing and analysis. The accuracy of any output depends on the quality and timeliness of the underlying data sources. The examples use the Python library yfinance, which retrieves data from Yahoo Finance. yfinance is an independent open-source project; it is not published by, affiliated with, or endorsed by Yahoo, and your use of the retrieved data is subject to Yahoo's terms of use. Check the terms that apply before you use the data – especially if you intend to share, publish, or commercially use any results. The code in this guide is released under the MIT License; the license text is in the repository (see "Code & Updates").

Liability

To the extent permitted by law, the author accepts no liability for decisions made on the basis of this guide or of the examples, data, and code it contains.

Author Disclosure

The author is not a licensed financial advisor, investment advisor, or broker-dealer in any jurisdiction.

Introduction

The Goal First method works on paper. At some point, it stops scaling.

Manually pulling 20 years of return data for 50 tickers, checking each against a benchmark year by year, filtering by loss years, ranking by average return – it's satisfying the first time. By the third iteration you're copy-pasting into spreadsheets at midnight and questioning your life choices.

Python fixes that. Not because it's magic, but because it's fast, repeatable, and – once the logic is in place – surprisingly straightforward. If you can describe your investment criteria clearly enough to explain them to another person, you can turn them into a screener. That's genuinely all it takes.

The starting point isn't the code. It's the same question as before: what are you selecting for, and what rules out a candidate? Once that's clear, the translation into Python is almost mechanical. The code in this guide follows the thinking – not the other way around.

> *You don't need to be a developer to build something useful. You need to know what you want.*

If you've read Goal First: A Smarter Way to Select Stocks, you already have the framework. This guide is the Python implementation of it. If you haven't, the first guide is the better place to start.

One more thing: if you'd rather have this built for you – customized to your goals, your data source, your metrics – feel free to reach out via Gumroad.

Part 1

From Logic to Code

The abstract pattern

Every screening framework, regardless of strategy, has the same three-part structure. Goal, constraints, ranking. That's it.

```
DEFINE goal          -> the metric you rank candidates by
DEFINE constraints   -> conditions that disqualify a candidate
RANK                 -> order what remains by the goal metric
```

In pseudocode – one level closer to actual code, but still readable as plain logic:

```
FOR each ticker in universe:
    metric = calculate_goal_metric(ticker)

    FOR each constraint in constraints:
        IF constraint is violated:
            EXCLUDE ticker
            BREAK

    IF ticker not excluded:
        ADD to candidates with metric

RANK candidates BY metric DESCENDING
RETURN candidates
```

This structure doesn't change between strategies. What changes are the inputs: which metric defines the goal, and which conditions trigger exclusion. The Python implementation reflects this directly – one core screening function, parameters you control.

> *The upside of starting here is that once you've built this once, adapting it to a different strategy is a matter of changing a few lines – not rewriting from scratch.*

Setting your parameters

Before writing a single function, answer three questions. Write them down. This is the specification. Everything else is execution.

WHAT IS YOUR GOAL?

The metric you will rank candidates by – or, as in the outperformance strategy, the property a candidate must have before it is ranked at all. Benchmark bias score, dividend yield, free cash flow yield. One metric. If you have two, decide which comes first. The screener can only rank by one thing – that constraint forces clarity that most manual approaches quietly avoid.

WHAT ARE YOUR CONSTRAINTS?

Conditions that disqualify a candidate regardless of how well it scores on the goal metric. Binary – pass or fail, no partial credit. Each constraint should protect against a specific failure mode. If you can't explain why a constraint is there, it probably shouldn't be.

WHAT IS YOUR UNIVERSE?

The list of tickers you will screen. A universe might be the S&P 500 constituents, a sector's major names, or a custom watchlist. Garbage in, garbage out: a well-built screener applied to a poorly chosen universe still produces poor results.

Parameter	Outperformance	Dividend
Goal	Bias score ≥ 0.60	Dividend yield ≥ 3.5%
Constraint 1	Loss year ratio ≤ 0.25	Loss year ratio ≤ 0.25
Constraint 2	Min. 10 years of history	Payout ratio ≤ 90%
Constraint 3	—	Min. 10 years of history
Ranking	Avg. annual return, descending	Avg. total return, descending
Universe	S&P 500 constituents	Custom dividend watchlist

Notice that both example strategies use loss year ratio as their first constraint. That is a choice, not a rule: constraints are chosen per strategy. In the examples shown, loss year ratio is the filter that bites first because it catches the most common failure mode – a stock whose goal metric looks fine but that loses money on a regular basis.

Two examples in pseudocode

Example 1 — Outperformance

Goal: stocks that consistently beat the S&P 500, ranked by average annual return.

```
FOR each ticker in universe:
    annual_returns    = get_annual_returns(ticker, years=20)
    benchmark_returns = get_annual_returns('^GSPC', years=20)

    # Goal: must beat benchmark consistently
    bias_score = count(annual_returns > benchmark_returns) / years
    IF bias_score < 0.60: EXCLUDE

    # Constraint 1: loss year ratio within personal threshold
    loss_year_ratio = count(annual_returns < 0) / years
    IF loss_year_ratio > max_loss_ratio: EXCLUDE

    # Ranking metric: average annual return (arithmetic mean)
    avg_return = mean(annual_returns)
    ADD to candidates

RANK BY avg_return DESCENDING
```

The bias score of 0.60 means the stock beat the S&P 500 in at least 60% of observed years. The loss year ratio threshold is personal, not fixed. A reasonable default for most investors is 0.25: if a stock was negative in more than one quarter of all observed years, it doesn't pass. Both are your parameters, not commandments.

Example 2 — Dividend

Goal: stocks with a sustainable dividend yield, ranked by average total return.

```
FOR each ticker in universe:
    div_yield      = get_dividend_yield(ticker)
    annual_returns = get_annual_returns(ticker, years=20)
    payout_ratio   = get_payout_ratio(ticker)

    # Constraint 1: loss year ratio within personal threshold
    loss_year_ratio = count(annual_returns < 0) / years
    IF loss_year_ratio > max_loss_ratio: EXCLUDE

    # Constraint 2: dividend must come from earnings
    IF payout_ratio > 0.90: EXCLUDE

    # Goal: yield above threshold
    IF div_yield < min_yield: EXCLUDE

    # Ranking metric: average total return (arithmetic mean)
    avg_return = mean(annual_returns)
    ADD to candidates

RANK BY avg_return DESCENDING
```

Same structure, different metrics. The payout ratio constraint catches the AT&T pattern from the first guide – a yield that looks attractive but is being funded by something other than stable earnings. One constraint, one failure mode eliminated.

Why this matters before the code

The pseudocode above could be written by anyone with a clear investment thesis and a quiet half hour. That's the point. The Python that follows is a direct translation of this logic – no architectural surprises, no hidden assumptions.

If the pseudocode is wrong, the code will be wrong in exactly the same way. That's actually an advantage: the error is visible, fixable, and in a place you control.

> *A screening framework is just your investment logic written in code. If your logic is clear, the code is straightforward. If your logic is fuzzy, the code will be too.*

Part 2

Data Quality

The screening logic from Part 1 is only as good as the data it runs on. This is less obvious than it sounds.

yfinance is reliable for most purposes. But “most purposes” is doing a lot of work in that sentence. For a framework that makes decisions based on 20 years of annual returns, small data problems compound into large output problems – quietly, without error messages. A screener that surfaces the wrong candidates because of bad data is worse than no screener at all, because it feels like it’s working.

Three specific problems are worth catching before the screener runs: invalid tickers, price distortions from splits, and zombie stocks. Each has its own detector. Together they form a data quality layer that sits between your universe and your screening logic.

And then, after all of that, you still look at the output yourself. More on that at the end.

The Ticker Validator

The simplest problem: a ticker that doesn’t work. This happens more often than expected – tickers change when companies merge, get acquired, or restructure. A list of index constituents from six months ago may already contain stale entries.

yfinance doesn’t raise an exception for an invalid ticker. It returns an empty DataFrame. If you don’t catch that, the screener silently skips the stock or – worse – produces a calculation error that propagates downstream.

```
def validate_tickers(tickers: list[str],
                     min_years: int = 10) -> dict:
    """
    Checks each ticker for data availability and
    minimum history length.

    Returns:
        {
          'valid': [...],
          'invalid': [...],
          'insufficient_history': [...]
        }
    """
```

The function returns three buckets: valid tickers with sufficient history, tickers that return no data at all, and tickers where data exists but doesn’t cover the minimum required period. A stock with only 3 years of history can’t be evaluated on a 20-year framework.

Ticker	Status	Years available
Stock A	✓ Valid	22

Ticker	Status	Years available
Stock B	✓ Valid	18
Stock C	✓ Valid	20
Stock D	× Invalid	—
Stock E	⚠️ Insufficient history	4
Stock F	✓ Valid	20
Stock G	× Invalid	—
Stock H	✓ Valid	15
Stock I	⚠️ Insufficient history	7
Stock J	✓ Valid	21

Six valid, two invalid, two with insufficient history. Run this before anything else. Anonymized example.

The Split Detector

yfinance adjusts historical prices for stock splits automatically. Most of the time, this works correctly. For older data – particularly splits that occurred before a certain date or in less liquid stocks – the adjustment can be wrong or missing entirely.

A mis-adjusted split looks like an extraordinary return. A 2-for-1 split that isn't correctly reflected in historical prices appears as a +100% gain in a single year. The screener sees a fantastic year; the investor sees a data artifact.

```
def detect_split_anomalies(ticker: str,
                           threshold: float = 0.8) -> list[dict]:
    """
    Flags years where annual return exceeds threshold in
    either direction and cross-checks against known split
    dates from yfinance corporate actions data.

    Returns list of flagged years with context.
    """
```

Any year with a return above +80% or below −80% is flagged for review. The function checks whether a known split event occurred in that year. If yes – plausible. If no – potential data artifact, inspect manually.

> *This doesn't remove flagged years automatically. It surfaces them. The decision to keep or exclude stays with you.*

The Zombie Detector

A zombie stock is one that is technically still trading but has no meaningful price activity. Low volume, price barely moving, effectively illiquid. These pass the ticker validator – the data exists – but they pollute the screener output because their return history is artificial.

The more dangerous variant is the reactivated zombie. These stocks sat dormant for years – price flat at or near $1, minimal volume, no real market – and then suddenly began trading again. A stock that moves from $1 to $3 in a year shows +200%. Mathematically correct. Practically meaningless, and lethal to any return-based ranking.

Japan Airlines is the textbook case. JAL filed for bankruptcy in January 2010 and was delisted shortly after. The airline relisted on the Tokyo Stock Exchange in September 2012 following a government-backed restructuring. Depending on how a data provider handles the gap, the historical price series either shows a clean break or stitches pre- and post-bankruptcy prices together. Either way, the calculated returns around the relisting are extreme – and tell you nothing useful about JAL as an investment after restructuring.

```
def detect_zombies(ticker: str,
                   price_floor: float = 2.0,
                   min_avg_volume: int = 100_000,
                   dormancy_threshold: float = 0.05) -> dict:
    """
    Detects two zombie patterns:

    1. Active zombie: trading but effectively illiquid
       -> avg daily volume below min_avg_volume

    2. Reactivated zombie: long dormancy period
       (price near floor, near-zero annual movement)
       followed by extreme return spike

    Returns classification and flagged periods.
    """
```

The reactivated zombie pattern specifically: the detector looks for multi-year periods where annual price movement stays below the dormancy threshold – less than 5% in either direction – followed by a year with an outsized return. That combination is the signature.

Ticker	Pattern	Flagged period	Note
Stock K	Active zombie	2019–2023	Avg. volume: 8,200
JAL (9201.T)	Reactivated zombie	Delisted 2010, relisted 2012	Dormancy + extreme return spike
Stock M	Active zombie	2021–2023	Price range: $0.80–$1.10

Three stocks flagged from a universe of 80. All passed the ticker validator. Anonymized example except JAL.

JAL would have ranked near the top of any return-based screener without the zombie detector – and told you nothing useful about the company as it actually exists today.

The one thing no detector replaces

Run all three. Fix what they flag. Then look at the output yourself.

This is not a disclaimer buried in the appendix. It’s a core part of the method.

A screener reduces a universe of hundreds of stocks to a shortlist of candidates. That's its job – and it does it well. What it can't do is know what you know. It doesn't know that a company on the list just announced a major restructuring. It doesn't know that a sector is facing regulatory headwinds that haven't shown up in the numbers yet. You do.

The final step is always a manual review of every candidate that makes the list. Not hours of research – a sanity check. Current news, recent filings, anything that would make you pause. The screener earns your attention. The decision stays with you.

> *A framework that removes judgment entirely is not a framework. It's a liability.*

Part 3

The Screener

The pseudocode from Part 1 described the logic. The data quality layer from Part 2 cleaned the input. Now the two come together.

The screener is a single function with configurable parameters. It takes a validated list of tickers, fetches their return history, applies the goal-first filter, and returns a ranked DataFrame. The structure is the same regardless of strategy – what changes are the parameters.

Setup

Three libraries cover everything this framework needs:

```
pip install yfinance pandas numpy
```

```
import yfinance as yf
import pandas as pd
import numpy as np
```

yfinance handles data fetching. pandas handles the return calculations and filtering. numpy handles the arithmetic. No further dependencies.

> A word on yfinance before you install it: the library is an independent open-source project that retrieves data from Yahoo Finance – with no official API and no affiliation with Yahoo. What you may do with the data is governed by Yahoo's terms of use, not by the library. For your own screener on your own machine this is usually uncritical. If you intend to share, publish, or commercially use the results, check the terms yourself first – they change, and this guide can't read them for you.

Fetching annual returns

yfinance returns daily adjusted closing prices. Annual returns have to be calculated from those prices – one year-end price divided by the previous year-end price, minus one.

```
def get_annual_returns(ticker: str,
                       years: int = 20) -> pd.Series:
    """
    Fetches adjusted closing prices and calculates
    calendar year returns.

    Returns pd.Series indexed by year (int),
    values as decimals e.g. 0.15 for +15%.
    """
```

The metrics in Part 4 also need the price series itself, not just annual returns. A second, smaller function covers that:

```
def get_prices(ticker: str,
               years: int = 20) -> pd.Series:
    """
    Fetches adjusted closing prices for the lookback period.

    Returns pd.Series indexed by date, chronological order.
    """
```

ADJUSTED PRICES

yfinance returns split- and dividend-adjusted prices by default. For the outperformance strategy this is what you want – total return including dividends. For a pure price-return comparison, pass auto_adjust=False. Know which one you're using before you interpret the output.

CALENDAR YEARS

The function returns one number per calendar year – January to December. This makes the bias score and loss year count intuitive and directly comparable across stocks. A stock that had a strong Q4 but a weak overall year still shows as a loss year. That's the correct behavior.

The core screening function

Every parameter has a default but none are fixed.

```
def screen(tickers: list[str],
           benchmark: str = '^GSPC',
           min_bias: float = 0.60,
           max_loss_ratio: float = 0.25,
           min_years: int = 10,
           years: int = 20) -> pd.DataFrame:
    """
    Applies goal-first screening logic to a validated
    ticker list.

    Parameters:
        tickers        - validated ticker list
        benchmark      - benchmark ticker (default: S&P 500)
        min_bias       - goal: min fraction of years beating benchmark
        max_loss_ratio - constraint: max fraction of years with negative return
        min_years      - constraint: minimum years of data required
        years          - lookback period in years

    Returns:
        pd.DataFrame sorted by avg_annual_return descending.
        Columns: ticker | bias_score | loss_year_ratio | avg_annual_return
    """
```

The function returns only candidates that met the goal threshold and passed all constraints, ranked by average annual return. Excluded tickers don't appear in the output.

What the output looks like

Anonymized example – 40 tickers in, 7 passed everything:

Ticker	Bias score	Loss year ratio	Avg. annual return
Stock A	0.75	0.10	+18.4%
Stock B	0.70	0.15	+16.1%
Stock C	0.65	0.20	+14.8%
Stock D	0.80	0.20	+13.9%
Stock E	0.60	0.25	+12.7%
Stock F	0.65	0.15	+11.3%
Stock G	0.60	0.20	+10.8%

33 tickers excluded. 7 candidates remain, ranked by average annual return. Anonymized example.

> *This is the shortlist. Not a buy list. The next step is yours.*

Adapting to a different goal

Switching from outperformance to dividends doesn't require rewriting the screener. It requires changing what you fetch and what you filter on.

For a dividend screener, the goal metric changes from the bias score to dividend yield. The constraints change to loss year ratio and payout ratio. The core function stays identical. The signature is in Part 5, Variant 1.

> *Once you've built one version of this, you've built all of them. The hardest part is the first one – and you've already done that.*

Part 4

Adding Metrics

The screener ranks candidates by average annual return. That's enough to produce a useful shortlist. But a shortlist is a starting point, not a conclusion.

Metrics are descriptive: they tell you more about the candidates that already passed, so your manual review is informed rather than blind. Three metrics cover most of what matters. Each is a standalone function that takes a return series and returns a single number.

Metric 1 — Average Annual Return

Already used in the screener. Worth making explicit as a standalone function because it appears in every strategy variant.

```
def average_annual_return(annual_returns: list[float]) -> float:
    """
    annual_returns: list of decimal returns
                    e.g. [0.15, -0.12, 0.23]
    returns: arithmetic mean as decimal
             e.g. 0.087 for 8.7%
    """
    return sum(annual_returns) / len(annual_returns)
```

Year 1: +15%. Year 2: −12%. Year 3: +23%. Average: 8.7%. Each year is treated as an independent observation. This is the right approach when comparing yearly slices across different stocks – you're measuring consistency of annual performance, not terminal wealth. Every year stands on its own.

Metric 2 — Maximum Drawdown

Maximum drawdown measures the largest peak-to-trough decline over the observed period. It's the answer to the question every investor eventually asks: what's the worst this stock has done, and could I have held through it?

```
def max_drawdown(prices: pd.Series) -> float:
    """
    prices: pd.Series of adjusted closing prices,
            chronological order

    returns: maximum drawdown as negative decimal
             e.g. -0.43 for 43% peak-to-trough decline
    """
```

A maximum drawdown of −43% means that at some point during the observed period, an investor who bought at the peak was sitting on a 43% loss before recovery. That's not a hypothetical – it's a lived experience for anyone who held through it.

Maximum drawdown pairs naturally with the loss year ratio. Loss year ratio tells you how often a stock was negative on an annual basis. Maximum drawdown tells you how bad the worst episode was. Both together give you a much more complete picture of what holding this stock actually feels like.

> *A stock with a low loss year ratio but a high maximum drawdown is one that tends to be fine – until it really isn't.*

Metric 3 — Benchmark Bias Score

Already used as the goal threshold in the screener. As a standalone metric it gives you more granularity – not just how often the stock beat the benchmark, but by how much in the years it did.

WHY CALENDAR YEARS, NOT ROLLING WINDOWS

Each calendar year has its own market character – a sector that runs hard, a macro theme that dominates, a rate environment that favors or punishes certain industries. These themes align with calendar years, not arbitrary 12-month windows.

Rolling windows create a specific distortion: a strong year that runs from, say, October to September gets split across two rolling periods. Both receive a positive contribution from the same underlying performance. The bias score counts it twice. The mirror image applies to weak years – one bad year becomes two negative rolling periods. Calendar years are discrete. Each year stands alone. A good year is counted once. A bad year is counted once.

```
def benchmark_bias(stock_returns: pd.Series,
                   benchmark_returns: pd.Series) -> dict:
    """
    stock_returns:     annual returns as pd.Series indexed by year
    benchmark_returns: annual returns as pd.Series indexed by year

    returns: {
        'bias_score': float,         # fraction of years beating benchmark
        'avg_outperformance': float  # avg excess return in years stock won
    }
    """
```

	Bias score	Avg. outperformance
Stock A	0.70	+9.2%
Stock B	0.70	+4.8%

Same consistency, different magnitude. The bias score shows both as equal. The metric surfaces the difference. Anonymized example.

Stock A beats the benchmark by almost twice as much in the years it wins. The screener output would show both as equally valid. This metric tells you which one is doing more work.

Keeping the architecture clean

Each metric is a function that takes a return series and returns a number. Functions with a single responsibility are easy to test, easy to replace, and easy to combine. Adding a new metric to the screener output is one line:

```
results['max_drawdown'] = results['ticker'].apply(
    lambda t: max_drawdown(get_prices(t))
)
```

The screener doesn't need to know what metrics you've attached. The metrics don't need to know anything about the screener. They're independent – which means you can extend the framework without breaking what's already working.

> *Start with the three metrics above. Add others when you have a specific question they don't answer. Build only what you need.*

Part 5

Adapting to Your Goal

The screener, the data quality layer, and the metrics are all built around one strategy so far – outperformance. That's intentional. Learning the framework on one concrete example is easier than learning it in the abstract.

The framework isn't an outperformance screener. It's a structure that works for any clearly defined investment goal. This section shows two strategy variants, each built from the same components, each requiring only a change of parameters and metrics.

The pattern, restated

Every variant follows the same sequence:

Step	Action
1	Define goal metric – what you rank by
2	Define constraints – what disqualifies a candidate
3	Fetch the right data – goal metric + constraint inputs
4	Apply the filter – same core logic
5	Rank and review – output + manual check

The only part that changes between strategies is steps 1 through 3. The rest is identical.

Variant 1 — Dividend

Goal: dividend yield of at least 3.5% as a threshold. Ranking is by average total return – price appreciation plus dividends, as in Guide 1.

CONSTRAINTS

Loss year ratio ≤ 0.25 – as in the outperformance variant. Payout ratio ≤ 0.90 – dividend must be funded by earnings, not by issuing debt or selling assets.

```
def screen_dividend(tickers: list[str],
                    min_yield: float = 0.035,
                    max_loss_ratio: float = 0.25,
                    max_payout_ratio: float = 0.90,
                    min_years: int = 10) -> pd.DataFrame:
    """
    Dividend variant of the core screener.

    Goal: dividend yield above min_yield
    Constraints: loss year ratio, payout ratio
    Ranking: average annual total return, descending
    """
```

The AT&T pattern from the first guide fails at the loss year ratio constraint. Four negative years in six (2017–2022) is a ratio of 0.67 – more than double the threshold. The payout ratio constraint catches the cases where the loss year history looks acceptable but the dividend is being sustained artificially.

Ticker	Dividend yield	Loss year ratio	Payout ratio	Avg. total return
Stock D	4.1%	0.10	55%	+11.2%
Stock B	4.8%	0.15	71%	+9.8%
Stock F	3.6%	0.15	80%	+9.1%
Stock C	4.3%	0.20	68%	+8.1%
Stock A	5.1%	0.10	62%	+7.4%
Stock E	3.8%	0.20	74%	+6.9%

35 tickers in, 6 passed. Highest yield in the full universe was 8.9% – it didn't pass constraint one. Ranked by total return, not by yield. Anonymized example.

Variant 2 — Low Volatility

Goal metric: annualized standard deviation of annual returns – lower is better. Ranked ascending: least volatile at the top.

CONSTRAINTS

Loss year ratio ≤ 0.25 – as in the other two variants. Maximum single-year drawdown ≤ 30% – a stock can have low average volatility but still have one catastrophic year; this constraint catches that.

```
def screen_low_volatility(tickers: list[str],
                          max_loss_ratio: float = 0.25,
                          max_single_drawdown: float = 0.30,
                          min_years: int = 10) -> pd.DataFrame:
    """
    Low volatility variant of the core screener.

    Goal metric: annualized standard deviation
    Constraints: loss year ratio, max single-year drawdown
    Ranking: std deviation ascending (lowest first)
    """
```

Standard deviation is an average – it smooths over individual years. A stock with mostly calm years and one −45% year can still show a moderate standard deviation. That single year is exactly the event that causes investors to sell at the worst moment. The constraint catches it explicitly rather than letting it hide in the average.

Ticker	Ann. std dev	Loss year ratio	Max single-year drawdown
Stock A	8.2%	0.10	−11%
Stock B	9.4%	0.15	−14%
Stock C	10.1%	0.10	−18%

Ticker	Ann. std dev	Loss year ratio	Max single-year drawdown
Stock D	11.3%	0.20	−22%
Stock E	12.0%	0.15	−19%
Stock F	12.8%	0.20	−27%
Stock G	13.4%	0.25	−25%
Stock H	14.1%	0.20	−29%

The lowest volatility stock in the full universe failed on loss year ratio. The second lowest had a max single-year drawdown of −38%. Anonymized example.

Building your own variant

If your goal is different – value, quality, a sector-specific metric – the construction process is the same.

Define the goal metric first. Then ask: what specific failure mode would undermine this goal? That failure mode becomes a constraint. Loss year ratio appears in all three variants shown here; whether it belongs in yours is decided by your goal. Fetch only the data you need. Keep the core screening function unchanged.

> *The framework is not the code. The framework is the thinking. The code is just what happens when the thinking is clear enough to write down.*

Conclusion

Build It Once, Use It Forever

The framework in this guide has four moving parts: a data quality layer that catches problems before they reach the screener, a core screening function that applies your goal-first logic consistently, a metrics layer that tells you more about the candidates that passed, and a manual review step that keeps you in the loop.

None of these parts are complicated in isolation. What makes them useful is that they work together – and that they're yours. A screener you built yourself, around your own investment objectives, with constraints you chose for reasons you understand, is a fundamentally different tool than one you downloaded or inherited. You know what it does and why. You know what it doesn't do. You can adapt it when your circumstances change – when your income needs shift, when your risk tolerance moves, when life asks something different of your portfolio.

> *The best investment tool is one you understand completely. Everything else is a black box with a nice interface.*

This guide has covered the outperformance strategy in depth and two further variants. The data quality layer applies to all of them. The metrics apply to all of them. The manual review applies to all of them. If you want to add a third strategy, the construction process is in Part 5. If you want to add a fourth metric, the architecture is in Part 4.

What this guide hasn't covered – intentionally – is portfolio construction, position sizing, rebalancing, and execution. Those are separate topics, each large enough for their own treatment. The screener tells you which candidates are worth considering. Everything that follows is a different decision.

One more thing. Building this framework takes time. Not because the code is difficult, but because the thinking behind it has to be yours – your goal, your constraints, your risk tolerance. Some investors find that process engaging. Others would rather skip it and have something working. If you're in the second group, feel free to reach out via Gumroad.

This guide provides a framework, not a formula. The logic is transferable – the decisions are yours.

Goal first. Constraints second. Build once, use forever.

The rest is execution.

Glossary

Method terms as in Goal First – Part One, plus the terms used in the code. Code names in parentheses.

Adjusted close

Historical price adjusted for splits and dividends so that returns are comparable over time. yfinance returns it by default.

Average annual return (avg_annual_return)

Arithmetic mean of calendar-year returns. Ranking metric of the outperformance and dividend strategies.

Benchmark bias score (bias_score)

Fraction of calendar years in which the stock beat the benchmark (^GSPC). Goal of the outperformance strategy; threshold here 0.60.

Calendar-year return

Return from 1 January to 31 December. The method counts in calendar years, not rolling windows – each year counts exactly once.

Constraint

Binary exclusion condition. Pass or fail; no partial credit. Each constraint protects against one failure mode.

DataFrame / Series (pandas)

Table and column structures in pandas. The screener returns a DataFrame with one row per candidate.

Loss year ratio (loss_year_ratio)

Share of calendar years with a negative return. Used as a constraint in all strategies shown; code default 0.25 (max_loss_ratio), freely adjustable.

Maximum drawdown (max_drawdown)

Largest peak-to-trough decline in the period, as a negative decimal (−0.43 = 43%).

Pseudocode

Logic written in program form without the syntax of any language. Readable by anyone; the step before Python.

Standard deviation, annualized

Measure of the dispersion of annual returns. Goal metric of the low-volatility variant; lower is better.

Stock split

Division of one share into several (2-for-1 halves the price). Unadjusted, a split looks like a price crash or jump.

Ticker

Exchange symbol (SNAP, 9201.T). ^GSPC is Yahoo Finance's symbol for the S&P 500.

Universe

The list of tickers to be screened – index constituents, a sector, a watchlist.

yfinance / pandas / numpy

The three Python libraries of the framework: data retrieval from Yahoo Finance, table processing, arithmetic.

Zombie stock

A stock that technically trades but has no real market (volume, price movement). Reactivated zombies produce extreme spurious returns after dormant periods.

About the Author

Leverage Thinking is the pen name of Laura Kruse-Stecher. Equity investor for 20+ years. Proprietary stock selection algorithms in live trading. Data integrity and backtesting infrastructure: Python, PostgreSQL/TimescaleDB. Economist; managing director and sole shareholder of Elcando GmbH. The practice mentioned in the introduction to the first guide was brought through the crisis and later sold.

Questions, comments, custom builds: via Gumroad.

Code & Updates

The code behind this framework – data quality layer, screener, and metrics from Goal First in Python – lives in a public repository:

https://github.com/Elcando-GmbH/goal-first

Corrections and updates to both guides are published there as well. Error reports are welcome – via Gumroad or directly in the repository. The code is released under the MIT License.

Imprint

Author: Laura Kruse-Stecher, writing as Leverage Thinking.

Legal notice and contact: leveragethinking.gumroad.com

This guide was produced with the help of AI tools for research, editing, and translation. The method, the content, the examples, and every decision are the author’s own.